



L-Università ta' Malta
Faculty of Engineering

Department of Industrial
& Manufacturing
Engineering

Faculty of Engineering

Department of Industrial and Manufacturing Engineering

2024/2025

MFE4211 – Applications of AI in Engineering

PROJECT PORTFOLIO

Tomas Indruch

2413852

Contents

1.	Assignment 1.....	3
1.1.	Part 1.....	3
1.2.	Part 2.....	5
1.3.	References	6
2.	Assignment 2.....	7
2.1.	Part 1.....	7
2.2.	Part 2 + 3 + 4 (preparation)	7
2.3.	Part 2 + 3 + 4	10
2.4.	References	11
3.	Assignment 3.....	12
3.1.	Part 1.....	12
	Image preparation	12
	Sobel edge detection	12
	Canny edge detection.....	12
	Contours.....	13
3.2.	Part 2.....	13
3.3.	Part 3.....	14
3.4.	References	14
4.	Assignment 4	15
4.1.	Part 1.....	15
4.2.	Part 2.....	15
4.3.	Part 3.....	16
4.4.	Part 4.....	17
4.5.	Part 5.....	18
4.6.	References	18

1. Assignment 1

1.1. Part 1

1) **Feed rate optimization**

The feed rate is the distance traveled in one revolution by the cutter and directly affects the machining time and surface finish of the product. A high feed rate can reduce machining time but may affect surface quality or tool life. On the other hand, a low feed rate improves surface quality but increases production time. GAs can help with finding the best feed rate by evaluating different combinations and selecting the one that balances speed, quality, and tool wear.

2) **Cutting speed optimization**

Cutting speed is critical for determining the efficiency of the machining process. Too high speed can lead to overheating and tool wear and poor dimensions accuracy, while too low speed may cause inefficient material removal rates. GAs can evaluate and test multiple cutting speeds to identify the optimal one. This not only maximizes material removal but also reduces heat generation, improving tool longevity and energy efficiency.

3) **Tool path optimization**

In CNC machining, the tool path defines the movement of the cutting tool. An inefficient tool path increases machining time and energy consumption. GAs can optimize the tool path by exploring different movement patterns to minimize travel time, ensure smooth transitions between cuts, and maximize cutting efficiency. Additionally, they can consider constraints such as collision avoidance and tool accessibility for complex geometries.

4) **Tool wear and tool life optimization**

Tool wear affects both product quality and production costs. GAs can analyze the relationships between cutting parameters and tool wear. By optimizing these parameters, GAs can extend tool life and reduce tool replacement costs. This also helps with better scheduling of maintenance controls and reducing total maintenance time.

5) **Cost and energy efficiency**

Electric motor energy consumption is an increasing concern in manufacturing. GAs can optimize cutting parameters to reduce power usage while maintaining high productivity. For example, it can calculate trajectories with least friction to overcome and keep all the possible motors moving so there's no need to overcome static friction, which is way higher than dynamic.

6) **Batch size**

In CNC machining, multiple objectives often need to be considered, GAs are particularly useful in solving such multi-objective problems. They can find the best possible placements of the parts within the given parameters so that most parts can be manufactured, but also help with waste reduction. For example, GAs can find part placements within raw material sheets to minimize waste while maximizing productivity.

7) **Fixture and clamping optimization**

Proper placement and clamping of workpieces are fundamental for machining accuracy, stability, and safety. GAs optimize the orientation and positioning of workpieces to ensure maximum stability and

minimal deformation during machining. This reduces stress on both the machine and the part, which is particularly beneficial for delicate materials or intricate geometries.

8) Reducing vibration and noise

Excessive vibration and noise during machining degrade surface quality and accelerate tool wear. GAs optimize cutting parameters and tool paths to minimize these issues. For example, they can adjust feed rates and cutting speeds to avoid resonance frequencies or reduce cutting forces. This leads to quieter operation, longer tool life, and improved product quality.

9) Adaptive control systems

In real-time CNC machining, GAs can be integrated into adaptive control systems. These systems monitor machining conditions and adjust parameters dynamically to maintain optimal performance. For example, GAs can modify the feed rate or cutting speed based on real-time tool wear data, temperature, energy consumption and other factors.

10) CNC machines position

For factories with multiple CNC machines, scheduling machines positions efficiently is critical. GAs can optimize their order and allocation to different machines, reducing idle time and ensuring balanced workloads across the factory.

11) Post machining inspection optimization

Although primarily focused on machining, GAs can also optimize post-machining inspection processes. They can determine the best inspection sequences, tool placements, and measurement strategies.

1.2. Part 2

First, we need to say, what must be the result of the CNC machining process. At the end we expect to get a manufactured part with desired properties at the least cost to make it in shortest time possible. But that comes with some difficulties like CNC motors max speed, temperature of the part or cutter life. So, what's basically our task to do is to find the best balance between material removal rate and costs to do so.

The code remains basically same only with a few modifications. People and their schedules would be replaced with only one machine with its parameters. However, the evaluation matrix, which we try to minimize, is still price. Machines properties are cutting speed, feed rate, depth of cut, spindle speed, purchasing cost, lifespan and batch size. There will be also a set of boundary conditions like time and energy costs (rent for the place), maximum temperature, minimal surface quality.

What we need to do is to create a function, which will give us random initial values of the machine parameters (similar to *staffplanning_to_hourlyplanning*). These parameters will act like genes and will be changed in the algorithm later in the code.

With initial genes can the loop start with selecting acceptable machines with **is_acceptable** and **select_acceptable** functions and filter all the machines that don't pass. These functions will consider fact, that the products temperature and surface roughness limits cannot be exceeded

After this filter comes fitness evaluation, which will assign each gene a **total_cost** value, depending on the parameters described above (slower speed → higher costs, but also wear etc.). Now we select the best genes from the whole population by using **select_best**, which selects the 10 % of the best performing parameters and creates new **parent_gen**. **Best_child** is the best performing gene from the generation.

Now there will be used combination and mutation functions to fill the 90 % missing in new population. We'll start with crossover **random_combine**. This function receives the **parent_gen**, selects randomly two of them, creates a mask, which has a crossover point from 2 to 7 (the length of the parent parameters). The child is created by taking the first parameters from the gene one (parent 1) and the rest from a gene number two (parent 2).

When a new parent population is created, every new parent is mutated by **mutate** function. Mutation works by selecting random parameters of the machine and randomly changing its value within the given limits. Number of parameters (genes), which will be mutated is defined by the user (**n_mutations**).

What we achieved now is that we have a new population. This process of selection, fitness evaluation, crossover and mutation is repeated as many times as there is the number of iterations. The result is the machine that has the lowest cost with the highest performance possible in the given time.

The part with plotting remains basically untouched and shows the progress of the algorithm in time. [1]



Figure 1: Generative algorithm

1.3. References

[1]

“UM IT Account - University of Malta,” *Um.edu.mt*, 2025. <https://www.um.edu.mt/vle/course/view.php?id=85928> (accessed Jan. 21, 2025).

2. Assignment 2

2.1. Part 1

Support Vector Machines (SVMs) are supervised learning algorithms used for classification and regression tasks. In two-dimensional space, SVMs work by finding a straight line that separates data points belonging to different classes. The line is positioned so that it creates the widest possible gap, or margin, between the two groups of points. This margin ensures that the separation is as clear as possible. The points closest to the line are called support vectors, and they play a key role in determining the position and orientation of the line. If the data points can be perfectly separated, the SVM will find the line that maximizes the margin while still correctly classifying all the points.

When the data is more complex and cannot be separated with a simple line, SVMs can operate in three or more-dimensional space by finding a flat plane that divides the points into distinct groups. To do this, the SVM transforms the data into this higher-dimensional space using a method called a kernel. For example, if the data in two dimensions forms concentric circles that cannot be separated by a line, the kernel transformation maps the points into three dimensions, where a flat plane can separate them. This process allows SVMs to handle non-linear relationships between data points. [3]

One of the most important hyperparameters is C, the regularization parameter. C describes how much the algorithm prioritizes minimizing classification errors versus maximizing the margin. A smaller C allows a softer margin, meaning the SVM can tolerate some misclassified points to create a simpler model that generalizes better. In contrast, a larger C focuses on minimizing errors, which might result in overfitting, especially if the data has noise or outliers.

Another key parameter is gamma, which controls how much influence a single training example has. Low gamma means more generalized data with smoother boundaries, but never overfits.

class sklearn.svm.SVC(*, C=1.0, kernel='rbf', degree=3, gamma='scale', coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=200, class_weight=None, verbose=False, max_iter=-1, decision_function_shape='ovr', break_ties=False, random_state=None)

Other key hyperparameters are the kernel type, tolerance, max iterations, class weight and others. [1, 2]

2.2. Part 2 + 3 + 4 (preparation)

The code starts with importing libraries necessary for running the code. Then there is a user defined point, which the algorithms will assign its class based on the training. The last step of the first stage is to load the csv file with data into the program.

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn import neighbors
from sklearn.svm import SVC
from mlxtend.plotting import plot_decision_regions

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report

# predict a class of this point -- user defines
point_for_prediction = [4.1, 6.4]

# load the document
data = np.genfromtxt('Book3.csv', delimiter=',')
```

Now comes the part where the data are separated into a group with position values and a corresponding group with labels. These two groups are then divided into another two parts – training and testing sets according to the ratio 80 % (train) / 20 % (test). Random state 42 is random number, but it ensures the same split every time I run it.

The last step before the algorithms is to predefine the hyperparameter. Results are dictionary type and performance results will be stored here.

```
# split data in two training sets
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.20, random_state=42)

# Variables to track best k and gamma
best_k = 1
best_gamma = 0.001
best_C = 0.01
best_accuracy_knn = 0
best_accuracy_svc = 0

# Store results for comparison
results = {"KNN": {}, "SVC": {}}
```

KNN algorithm is a loop to find the best k – parameter. It starts with assigning the classifier to K-nearest neighbor with k going from 1 to 50. Then the classifier fits the train data. The algorithm is now trained, and we can run the testing part, where the algorithm predicts classes of the test set of points in a plane. Then accuracy is calculated by comparing correctly predicted classes with incorrectly predicted. And to find the best k – parameter, there's an if condition saying: if the accuracy is higher then the best score so far, remember the k – value.

```
# KNN algorithm
for k in range(1, 50): # Test different k values
    clf_knn = neighbors.KNeighborsClassifier(n_neighbors=k)
    clf_knn.fit(X_train, y_train) # Use only training data
    y_pred = clf_knn.predict(X_test) # Predict on test data
    accuracy = accuracy_score(y_test, y_pred)

    if accuracy > best_accuracy_knn:
        best_accuracy_knn = accuracy
        best_k = k
```

When the loop is finished and the best k – parameter is found, the code is run one more time and produces data about decision regions, classifier metrics, classification report and accuracy, which are then printed and plotted. The performance scores will be saved for later comparison.

```

# Final KNN with best k
clf_knn = neighbors.KNeighborsClassifier(n_neighbors=best_k)
clf_knn.fit(X_train, y_train)
y_pred_knn = clf_knn.predict(X_test)

# Store KNN results
results["KNN"]["Accuracy"] = accuracy_score(y_test, y_pred_knn)
results["KNN"]["Precision"] = precision_score(y_test, y_pred_knn, average='weighted')
results["KNN"]["Recall"] = recall_score(y_test, y_pred_knn, average='weighted')
results["KNN"]["F1-Score"] = f1_score(y_test, y_pred_knn, average='weighted')

# Plotting decision region
plot_decision_regions(x, y, clf=clf_knn, legend=2)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('KNN with K=' + str(best_k))
plt.ylim((6, 6.5))
plt.xlim((4.02, 4.23))
plt.show()

print("Best KNN k:", best_k)
print("Accuracy KNN:", best_accuracy_knn)
print("Confusion Matrix KNN:\n", confusion_matrix(y_test, y_pred_knn))
print("Classification Report KNN:\n", classification_report(y_test, y_pred_knn))

```

Now the same process is repeated only with one difference and that is a different classifier, which is SVC with gamma and C parameters. Basically, the code is the same as KNN only there are two loops for tuning and SVC function with C and gamma parameters.

```

# SVC algorithm
for gamma in np.linspace(0.001, 100, 100): # Test different gamma values
    for C in np.linspace(0.01, 100, 100): # Test different C values
        clf_svc = SVC(gamma=gamma, C=C)
        clf_svc.fit(X_train, y_train)
        y_pred = clf_svc.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)

        if accuracy > best_accuracy_svc:
            best_accuracy_svc = accuracy
            best_gamma = gamma
            best_c = C

# Final SVC with best gamma
clf_svc = SVC(gamma=best_gamma, C=best_c)
clf_svc.fit(X_train, y_train)
y_pred_svc = clf_svc.predict(X_test)

# Store SVC results
results["SVC"]["Accuracy"] = accuracy_score(y_test, y_pred_svc)
results["SVC"]["Precision"] = precision_score(y_test, y_pred_svc, average='weighted')
results["SVC"]["Recall"] = recall_score(y_test, y_pred_svc, average='weighted')
results["SVC"]["F1-Score"] = f1_score(y_test, y_pred_svc, average='weighted')

# Plotting decision region
plot_decision_regions(x, y, clf=clf_svc, legend=2)
plt.xlabel('X')
plt.ylabel('Y')
plt.title('SVC with Gamma=' + str(round(best_gamma, 2)) + ' and C=' + str(best_c))
plt.ylim((6, 6.5))
plt.xlim((4.02, 4.23))
plt.show()

print("Best SVC gamma:", best_gamma)
print("Accuracy SVC:", best_accuracy_svc)
print("Confusion Matrix SVC:\n", confusion_matrix(y_test, y_pred_svc))
print("Classification Report SVC:\n", classification_report(y_test, y_pred_svc))

```

After all the stuff is printed, it's time to compare performances. This loop will take data from results dictionary, compare them and write who the winner is.

```
# Compare and Print Results
print("----- Comparison Results -----")
for metric in ["Accuracy", "Precision", "Recall", "F1-Score"]:
    knn_value = results["KNN"][metric]
    svc_value = results["SVC"][metric]

    if knn_value > svc_value:
        winner = "KNN"
    elif svc_value > knn_value:
        winner = "SVC"
    else:
        winner = "Tie"

    print(f"{metric}: KNN = {knn_value:.2f}, SVC = {svc_value:.2f} --> Winner: {winner}")
```

The vera last thing to do is to predict the values of the pre-defined point.

```
# Predict the new point
print("\n----- Point Label Prediction -----")
prediction_knn = clf_knn.predict([point_for_prediction]) # Wrap the point in a list
print("Predicted KNN for point", point_for_prediction, ":", prediction_knn)

prediction_svc = clf_svc.predict([point_for_prediction])
print("Predicted SVC for point", point_for_prediction, ":", prediction_svc)
```

2.3. Part 2 + 3 + 4

This is an example of how the results should look like at the end.



Figure 2: Results from KNN and SVM

Best $k = 13$, best gamma = 1.01 best $C = 46.47$, in comparison is the winner KNN algorithm (where it won in Accuracy and Recall criteria and the rest is tie).

Plus, the predicted class of a given value is written at the bottom.

2.4. References

[1]

“1.4. Support Vector Machines,” *scikit-learn*, 2024. <https://scikit-learn.org/1.5/modules/svm.html>

[2]

“UM IT Account - University of Malta,” *Um.edu.mt*, 2025. <https://www.um.edu.mt/vle/course/view.php?id=85928> (accessed Jan. 21, 2025).

[3]

“Support Vector Machine (SVM) in 2 minutes,” *www.youtube.com*. https://www.youtube.com/watch?v=_YPScrckx28

3. Assignment 3

3.1. Part 1

Image preparation

```
import cv2

# Read the original image
img = cv2.imread('test2.jpg')

# let's downscale the image using new width and height
down_width = 800
down_height = 600
down_points = (down_width, down_height)
resized_down = cv2.resize(img, down_points, interpolation= cv2.INTER_LINEAR)

# Convert to grayscale
img_gray = cv2.cvtColor(resized_down, cv2.COLOR_BGR2GRAY)
```

First is loaded a computer vision library and an image. Then the image is scaled down to predefined pixels for faster and easier recognition and that the image doesn't consume too much computation power when being processed. It is also converted to grayscale, because recognition patterns are trained on it.

```
# Blur the image for better edge detection
img_blur = cv2.GaussianBlur(resized_down, (3,3), 0)
```

Then the image is blurred for easier edge detection and noise reduction with Gaussian kernel with the size of 3x3 pixels, because sharp edges can negatively interfere with the algorithm.

Sobel edge detection

```
# Sobel Edge Detection
sobelx = cv2.Sobel(src=img_blur, ddepth=cv2.CV_64F, dx=1, dy=0, ksize=5) # Sobel Edge Detection on the X axis
sobely = cv2.Sobel(src=img_blur, ddepth=cv2.CV_64F, dx=0, dy=1, ksize=5) # Sobel Edge Detection on the Y axis
sobelxy = cv2.Sobel(src=img_blur, ddepth=cv2.CV_64F, dx=1, dy=1, ksize=5) # Combined X and Y Sobel Edge Detection
```

Sobel function detects color gradient in the blurred image. Dx and dy say in which direction should be the derivative done.

The most important part is ksize, which says the size of a kernel (window) is 5x5 pixels and the gradient is calculated here. Larger ksize captures edges over a larger area but might smooth over finer details.

Then pictures are shown for only x axis detection, then y-axis and then combined.

Canny edge detection

```
# Canny Edge Detection
edges = cv2.Canny(image=img_blur, threshold1=150, threshold2=175) # Canny Edge Detection
```

Canny function decides which of all edges are really edges and which are not. Any edges with gradient more than threshold2 are sure to be edges and those below threshold1 are sure to be non-edges. Those who lie between these two thresholds are “classified” edges. When they are connected to "sure-edge" pixels, they are considered an edge.

Contours

```
# apply binary thresholding
ret, thresh = cv2.threshold(img_gray, 20, 100, cv2.THRESH_BINARY)
```

The grayscale image is converted to binary image (thresh). Pixels with greater value than 20 are set to 100 (white) and lower to 0 (black) so then the image is composed only of black and white pixels (this is part of image preparation)

```
# detect the contours on the binary image using cv2.CHAIN_APPROX_NONE
contours, hierarchy = cv2.findContours(image=thresh, mode=cv2.RETR_TREE, method=cv2.CHAIN_APPROX_SIMPLE)
```

Function findContours takes the binarized image and looks for contours. It retrieves all contours and builds a hierarchical tree of contours nested into other contours. Method used applies strategy to store only endpoints of horizontal, vertical and diagonal lines to reduce memory.

```
image_copy = img_gray.copy()
cv2.drawContours(image=image_copy, contours=contours, contourIdx=-1, color=(255, 255, 255), thickness=2, lineType=cv2.LINE_AA)
```

DrawContours function takes contours detected in the previous step and draws them into a copy of that image. ContourIdx just says that all contours should be drawn. Color is in format BGR, thickness is pixel width if the line and lineType means it's anti-aliasing for smoother appearance. [1]

3.2. Part 2

1. *Describe the various parameters available for image preparation:*
 - a. Resizing down the image to the set number of pixels for height and width.
 - b. Method for resizing down – weighted average value (INTER_LINEAR) can be switched to INTER_NEAREST or INTER_CUBIC
 - c. The Gaussian blur can be done on a different square (for example 5x5) or can be used another method like medianBlur
 - d. Image thresholding can be also changed depending on the picture to make the edges most visible
2. *Select the best parameters for image preparation for the provided image:*
 - a. Down_width = 400
 - b. Down_height = 887 (the image is already pretty small)
 - c. Interpolation – linear
 - d. Gaussian blur
 - i. Kernel size 3x3
 - ii. Standard deviation 0
3. *Select the Best parameters for canny edge and sobel edge detection:*
 - a. Sobel – ksize = 7
 - b. Canny
 - i. Threshold1 = 140
 - ii. Threshold2 = 500
4. *Compare the performance of canny edge with sobel edge detection:*

- a. Sobel detection didn't suppress noise in the picture much but there was at least visible the middle symbol. The symbol wasn't super clear, but it will be possible to get an orientation from it. Produces thicker edges and detects perfectly general edges.
- b. Canny detection couldn't get the middle symbol clear without too much noise present. So, I tuned it to suppress the noise as much as possible. So, it draws holes and the object outlines perfectly and thin but the symbol was lost. Also, it was possible to delete the noise in the form of dust stains or light.
- c. So, if I needed the symbol for orientation I would use but if not I would use Canny, because it provides clear and thin edges without noise

3.3. Part 3

Edge-detected images focus on the outlines and boundaries of objects, making it easier to analyze shapes, features, and text. I would use Convolutional Neural Networks (CNNs) to detect and recognize objects based on these images. The system would be trained on labeled data, learning to recognize lines, curves, patterns, corners, and text, and then match these features to the correct product or object.

The CNN would be designed to identify edges and patterns even when objects are partially hidden, scratched, or incomplete. It could still analyze the image and predict what the object is. For example, it could identify car license plates and brands in parking lots, helping police find illegally parked or stolen cars. In supermarkets, self-checkouts could use this system to identify products on a conveyor belt without needing to scan barcodes.

Mechanics could use it to locate lost tools or parts by simply scanning their workspace, while warehouses could rely on it to sort packages by size and type. At airports, the system could help security staff by spotting dangerous items like knives or firearms in luggage.

3.4. References

[1]

“UM IT Account - University of Malta,” *Um.edu.mt*, 2025.
<https://www.um.edu.mt/vle/course/view.php?id=85928>

4. Assignment 4

4.1. Part 1

I decided to choose an automotive industry, VW concern exactly. All the materials and components are delivered to its factory by the producers or third parties, The factory then assembles the components into the product (car) and then VW delivery section takes care for distributing the cars to dealers.

4.2. Part 2

1. Suppliers

Role: Provide raw materials and components to the factory.

- Deliver components on schedule (e.g., engines, tires, electronics).
- Maintain consistent quality and address any defects.
- Communicate shipping updates and inventory availability.

2. Manager

Role: Oversees the entire production and logistics process.

- Plan production and work schedules and coordinate between departments.
- Address escalated issues from other roles.
- Optimize resource allocation and ensure targets are met.
- Communication with everybody

3. Maintenance Engineer

Role: Ensures machinery and systems function efficiently.

- Perform regular maintenance and repairs on factory equipment.
- Troubleshoot technical problems that affect production.
- Coordinate upgrades or replacements of machinery.

4. Operator

Role: Manages machinery and oversees specific production processes.

- Run production equipment according to specifications.
- Monitor output quality and report anomalies.
- Adjust machinery settings to optimize performance.

5. Worker

Role: Performs manual tasks during production.

- Assemble components and complete production tasks.
- Follow safety procedures and maintain workspace cleanliness.
- Report issues to operators or supervisors.

6. Admin/Developer

Role: Maintains digital systems and supports communication platforms.

- Develop and manage the chatbot
- Troubleshoot IT issues affecting communication or automation.
- Ensure cybersecurity and system integrity.

7. Drivers (Logistics)

Role: Transport materials to the factory and deliver finished cars to dealers.

- Safely transport goods and vehicles as per the schedule.
- Report delays or issues during transit.
- Ensure proper handling to prevent damage.

8. Dealers

Role: Sell cars to end customers and provide market feedback.

- Receive vehicles from the delivery section and inspect quality.
- Communicate inventory needs and market demands to VW.
- Address customer concerns and warranty issues.

9. Customers

Role: End-users of VW vehicles.

- Asks for vehicle availability and its features
- Provide feedback on product quality and performance.
- Engage in after-sales services (repairs, warranties).

4.3. Part 3

1. Sensors – slot availability in warehouse, temperature, pressure, wear of equipment
2. Sheets – inventory status, transportation, production and maintenance schedule,
3. Emails – correspondence, ordering

4.4. Part 4

{"role": "Manager", "content": "What is the current status of production?"},

{"role": "assistant", "content": "The production schedule is on track, with 70% of tasks completed for today. The remaining tasks are expected to finish by [time]."}

{"role": "Worker", "content": "What should I do if I find a defective component?"},

{"role": "assistant", "content": "Report the defect to the operator immediately. They will evaluate the issue and decide whether to replace the component or involve a supervisor."}

{"role": "Supplier", "content": "Can you confirm the status of my material shipment?"},

{"role": "assistant", "content": "Your shipment is in transit and expected to arrive at the factory by [time]. Tracking ID: [ID]. Please let us know if you need further assistance."}

{"role": "Operator", "content": "The machine is showing an error. What should I do?"},

{"role": "assistant", "content": "Stop the machine and report the error to the maintenance engineer immediately. Do not attempt to restart until authorized."}

{"role": "Maintenance Engineer", "content": "Is there any scheduled maintenance for today?"},

{"role": "assistant", "content": "Yes, maintenance is scheduled for the CNC machine at 2:00 PM. Please prepare the necessary tools and spare parts."}

{"role": "Admin/Developer", "content": "Can you update me on the chatbot performance?"},

{"role": "assistant", "content": "The chatbot is operating normally, with no detected issues. Last update applied: [date]. Let me know if you need system logs or further details."}

{"role": "Driver (Logistics)", "content": "Where is my next delivery destination?"},

{"role": "assistant", "content": "Your next delivery is to [Dealer Name, Location]. Estimated arrival time: [time]. Please confirm receipt upon delivery."}

{"role": "Dealer", "content": "Are there any new vehicles available for delivery?"},

{"role": "assistant", "content": "Yes, new vehicles are available for delivery. Models include [list]. Expected delivery date is [date]. Please confirm your inventory needs."}

{"role": "Customer", "content": "Can you tell me if Model X is available?"},

{"role": "assistant", "content": "Model X is currently available in [colors and configurations]. You can visit your nearest dealer or book online for delivery."}

{"role": "Manager", "content": "Are there any delays in material deliveries today?"},

{"role": "assistant", "content": "Yes, there is a delay in the delivery of electronic components from Supplier A. The revised ETA is [time]. We are coordinating to minimize the impact."}

[1]

4.5. Part 5

Query: What are the current production priorities for today?
Expected response: The current priorities are completing the assembly of ID-7 units and preparing shipments for Dealer B. Quality checks on finished goods are also scheduled.

Query: Machine A is too noisy and slow, is there maintenance planned?
Expected response: Yes, maintenance for Machine A is scheduled for [time]. Please avoid using the machine until the issue is resolved. Notify the maintenance team immediately if the problem worsens.

Query: How many storage units need to be filled with the ID-7 model?
Expected response: A total of [number] storage units need to be filled with the ID-7 model today. Current progress: [percentage] complete.

Query: What is the current status of my delivery?
Expected response: Your delivery is on route and expected to arrive at [location] by [time].
Tracking ID: [ID]. Please contact logistics if you need further assistance.

Query: Can you confirm if the car I ordered is ready for delivery?
Expected response: Yes, your car is ready for delivery. It will be dispatched to [Dealer Name, Location] by [date]. Please confirm the final delivery details.

4.6. References

[1]

“UM IT Account - University of Malta,” *Um.edu.mt*, 2025.
<https://www.um.edu.mt/vle/course/view.php?id=85928>